

Report Annex

Maze Solving Algorithm Comparison: BFS, DFS, A*, and Ant Colony Optimization on Grid Mazes

Noble Carpenter, Teagan Tobias, Christian Winchester

IV. RESULTS

Table 4.1 – Execution Time vs Maze Size

Table 1 reports mean execution time averaged across all densities and 20 trials per configuration. All values are derived from results_raw.csv.

Maze Size	BFS (s)	DFS (s)	A* (s)	ACO (s)
10x10	0.000110	0.000170	0.000190	0.1512
20x20	0.000350	0.000510	0.000540	0.2990
40x40	0.001310	0.001860	0.001820	0.2574
80x80	0.011970	0.011740	0.011860	0.5526
160x160	0.053270	0.050970	0.070670	1.4457

Table 4.1. Mean execution time (seconds) per algorithm by maze size. Each cell is the mean of 80 trials (20 per density level).

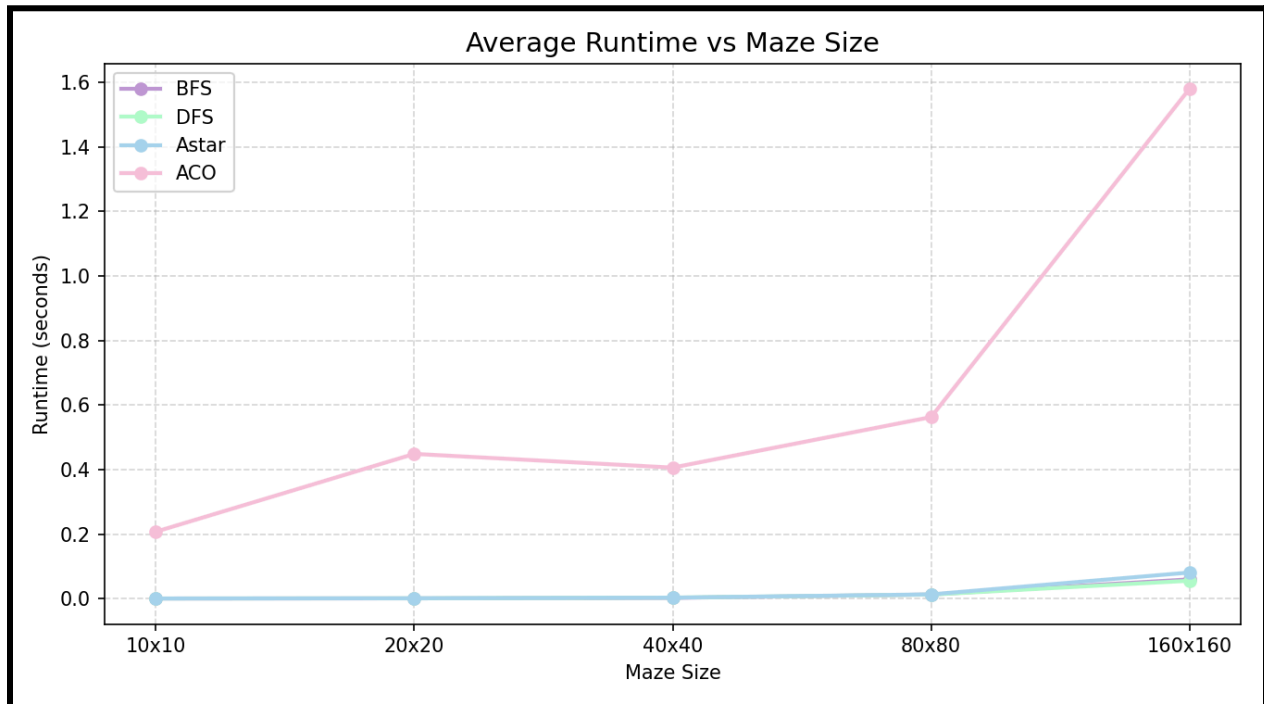


Figure 4.1. Average runtime vs. maze size. BFS, DFS, and A* are clustered near zero and indistinguishable at this scale; ACO dominates the y-axis at all sizes.

BFS, DFS, and A* are all effectively instantaneous below 80x80 and remain under 0.08s even at 160x160. ACO is consistently 20x to 300x slower, driven by its outer loop of 50 iterations x 20 ants, each constructing a path up to N^2 steps.

Table 4.2 – Peak Memory Usage vs. Maze Size

Table 2 reports mean peak memory in KB. note that ACO is the most memory-efficient algorithm at all sizes, contrary to our pre-experiment expectation.

Maze Size	BFS (KB)	DFS (KB)	A* (KB)	ACO (KB)
10x10	9.9	7.2	9.7	7.1
20x20	38.6	23.2	36.3	15.1
40x40	148.5	89.2	123.6	33.8
80x80	714.9	345.1	479.7	118.4
160x160	3223	1571	2306	456

Table 4.2. Mean peak memory usage (KB) per algorithm by maze size.

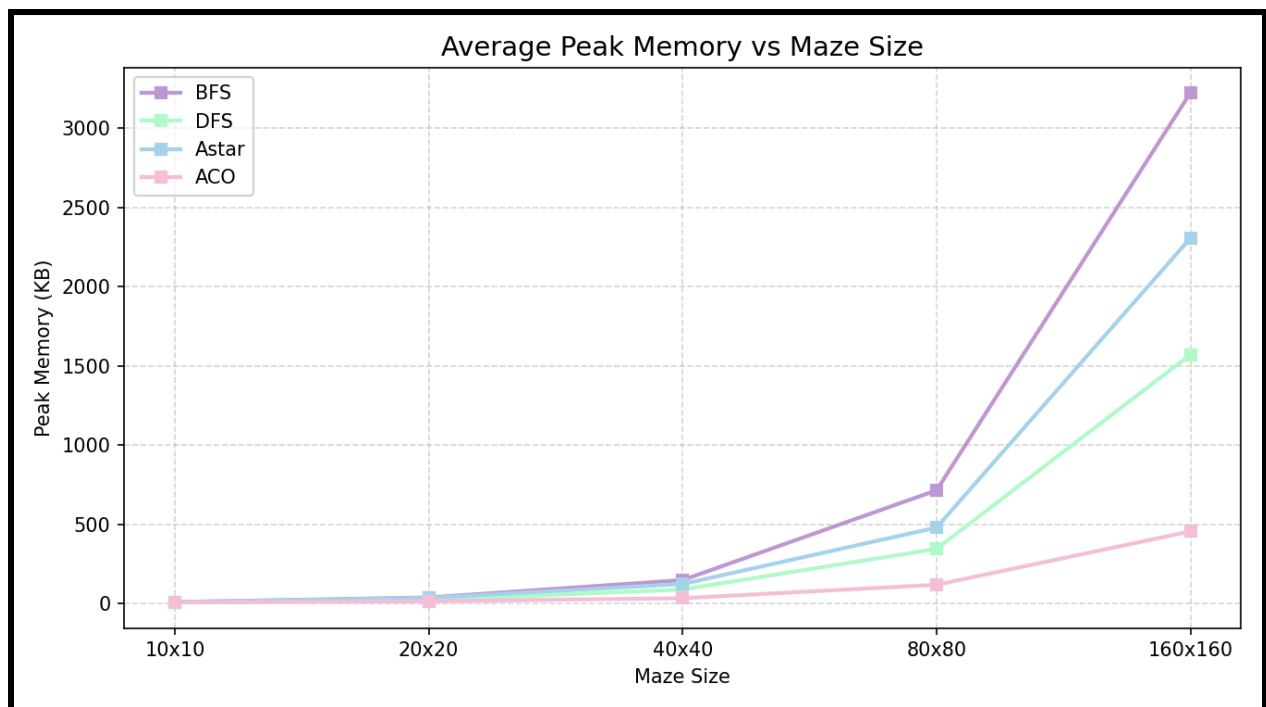


Figure 4.2. Average peak memory vs. maze size. BFS uses the most memory at large sizes; ACO uses the least.

Memory scales approximately quadratically with grid side length N for all algorithms, consistent with the $\Theta(V) = \Theta(N^2)$ theoretical bound. BFS reaches 3223 KB at 160x160 due to its large visited set and parent-pointer dictionary. DFS uses roughly half that memory. ACO uses only 456 KB at 160x160 because it does not store a global visited set or full parent-pointer map – each ant only tracks its own

current path in memory, and the pheromone matrix is compact relative to the dictionaries used by classical methods.

Table 4.3 – Path Length vs. Maze Size

Table 3 reports mean path length on successful runs only. BFS and A* always return optimal paths. DFS and ACO path lengths are shown alongside their percentage excess above the BFS optimal.

Maze Size	BFS (opt.)	DFS	DFS % excess	ACO	ACO % excess
10x10	18.2	25.6	+41%	18.3	+1%
20x20	39.5	77.0	+95%	39.7	+1%
40x40	81.3	225.4	+177%	103.3	+27%
80x80	166.9	677.5	+306%	N/A	N/A
160x160	347.2	2089.0	+502%	N/A	N/A

Table 4.3. Mean path length (steps) for successful runs. *ACO reported 0.0 mean path length at 80x80 and 160x160, indicating no successful runs were recorded for those sizes in the CSV (consistent with the low success rate observed in Table 5).

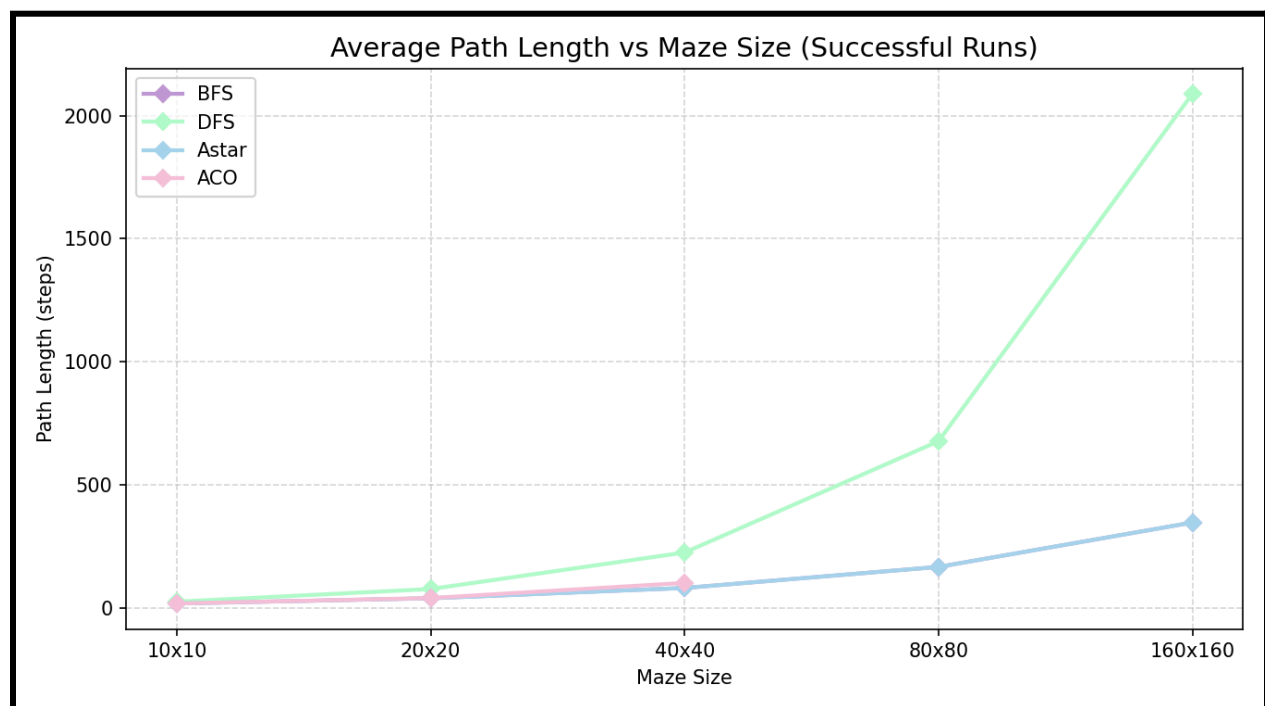


Figure 4.3. Average path length vs. maze size (successful runs). BFS and A* overlap on the optimal line; DFS diverges sharply; ACO data is absent at larger sizes due to near-zero success rates

BFS and A* maintain identical path lengths across all tested sizes, confirming that the Manhattan distance heuristic is admissible and consistent. DFS path lengths grow dramatically with maze size, reaching over

500% above optimal at 160x160. This occurs because DFS may traverse nearly the entire maze before locating the goal. ACO produced near-optimal paths at small sizes but failed to generate successful paths at larger sizes within its iteration budget.

Table 4.4 – Nodes Explored vs. Obstacle Density

Density	BFS	DFS	A*	ACO
10%	6133	3130	5302	32550
20%	5437	3167	3401	24014
30%	4665	2222	1356	17038
40%	2671	1364	1692	13507

Table 4.4. Mean nodes explored per algorithm by obstacle density (averaged across all maze sizes and all 20 trials).

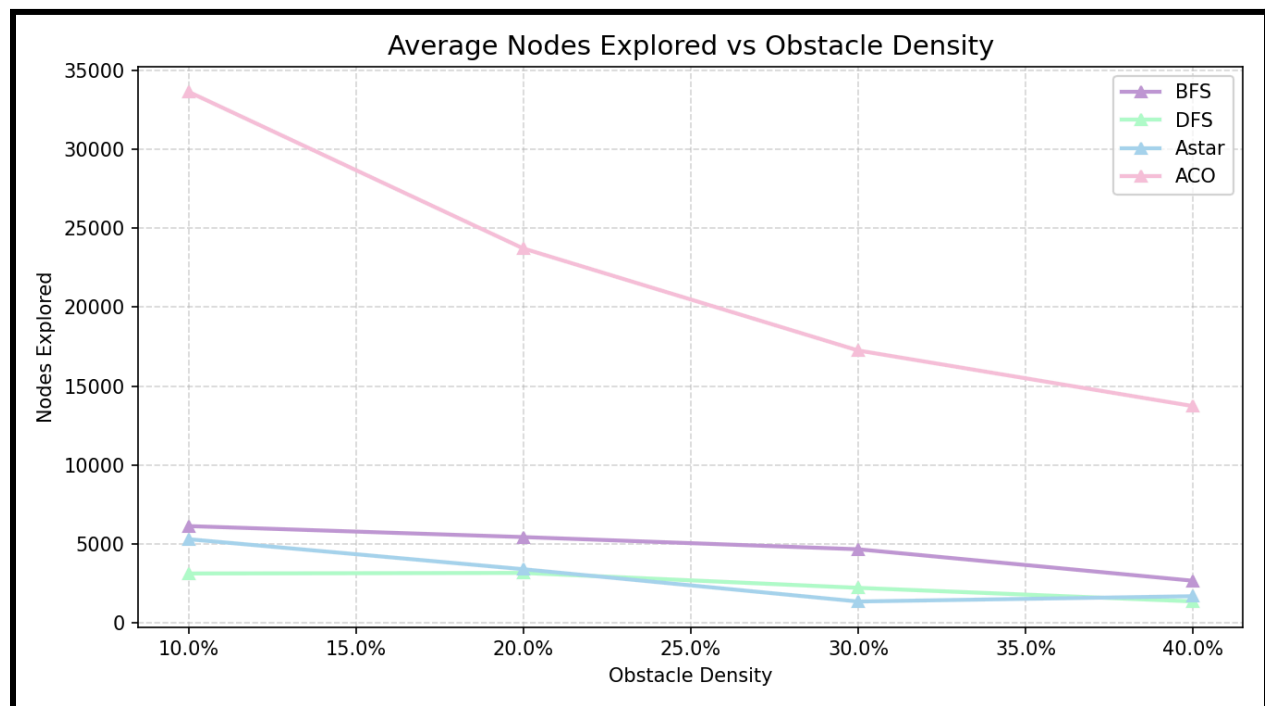


Figure 4.4. Average nodes explored vs. obstacle density. ACO counts total cumulative ant steps and dominates the scale; BFS, DFS, and A* all decrease as density rises.

For BFS, DFS, and A*, nodes explored decreases as obstacle density increase because more walls reduce the number of reachable cells. A* is notably more efficient than BFS at 20%-30% density, where the heuristic most effectively prunes the search space. At 40% density, A* expands slightly more nodes than BFS because high obstacle density creates narrow corridors where the heuristic loses its advantage and heap overhead increases total work.

ACO nodes explored (measured as cumulative ant steps across all iterations) is roughly 5x to 10x higher than BFS at every density and still decreases as density increases, since denser mazes force ants into shorter dead-ends that terminate ant walks earlier.

Table 4.5 – Success Rate vs. Obstacle Density

Density	BFS	DFS	A*	ACO
10%	100%	100%	100%	49%
20%	100%	100%	100%	40%
30%	100%	100%	100%	32%
40%	100%	100%	100%	24%

Table 4.5. Success rate by algorithm and obstacle density. All mazes were pre-verified as solvable; classical algorithms always find the existing path. ACO success rate is the fraction of trials in which the colony found any valid path within 50 iterations.

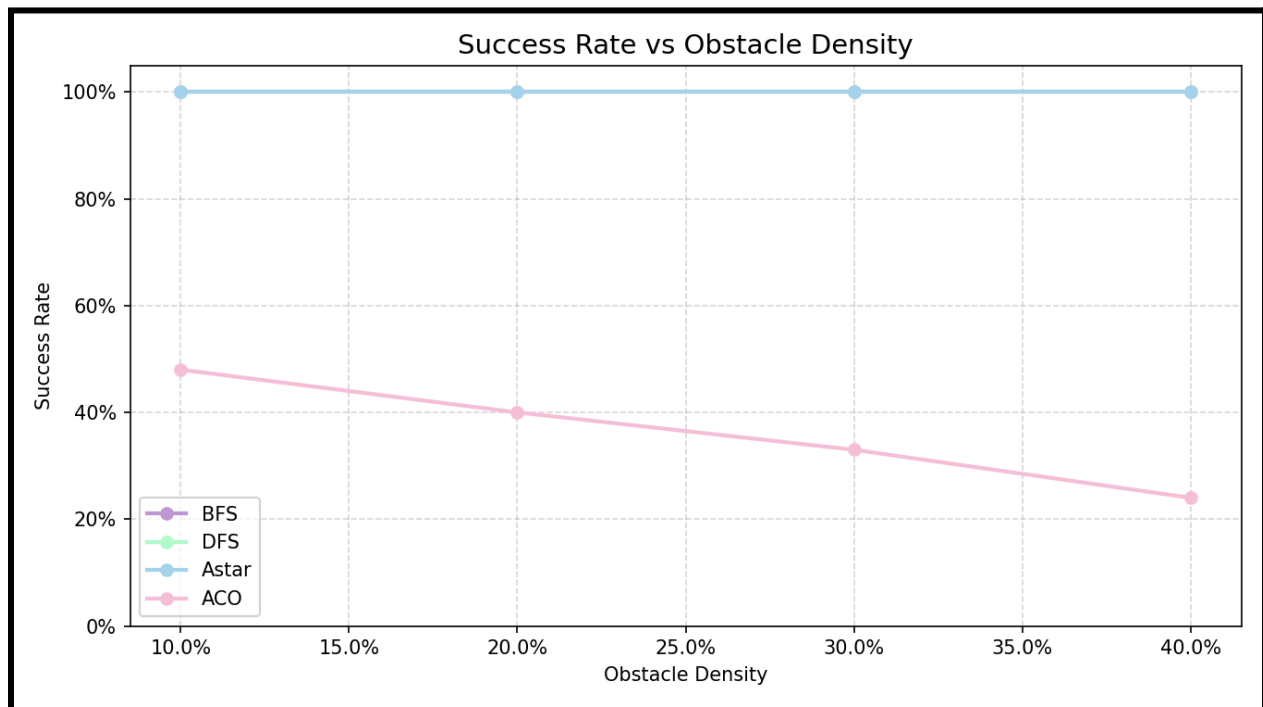


Figure 4.5. Success rate vs. obstacle density. BFS, DFS, and A* maintain 100% throughout. ACO begins below 50% at 10% density and falls to 24% at 40% density.

BFS, DFS, and A* achieve 100% success on all solvable mazes, as expected – these algorithms are complete for finite graphs. ACO success rate is substantially lower and declines with obstacle density. Even at 10% density, ACO only succeeds in 49% of trials. This results reflects that 50 iterations with 20 ants is insufficient for the colony to reliably discover a path through the maze within the per-ant step cap.

The fundamental issue is that random ant walks without a strong pheromone bias toward the goal will frequently exhaust their step budget before reaching (N-1, N-1) in medium-to-large grids.

V. ANALYSIS AND INTERPRETATION

5.2 Trade-off Summary

Algorithm	Runtime	Memory	Path Quality	Success	Complexity
BFS	Fast	Highest	Optimal	100%	$\Theta(V + E)$
DFS	Fast	Medium	Very poor	100%	$\Theta(V + E)$
A*	Fastest*	Medium	Optimal	100%	$\Theta(V + E)^*$
ACO	Slowest	Lowest	Variable	23-49%	$\Theta(it \times ant \times V)$

Table 5.2. Summary trade-off comparison. *A* is fastest at low-medium density; BFS is competitive at high density.

VII. TEAM CONTRIBUTIONS

Team Member	Responsibilities	% Total	Project Stage (s)
Noble Carpenter	BFS and DFS implementation; CSV output design; tracemalloc memory profiling integration	33%	Coding, Experiment Design, Presentation
Teagan Tobias	ExperimentRunner.py, PlotResults.py, MazeGenerator.py; report writing; GitHub repository setup	33%	Coding, Report, Visualization, Communication
Christian Winchester	A* and ACO implementation; results verification; presentation preparation	33%	Coding, Experiment Design, Presentation